

Coordinate-wise SVRG for Low Signal-to-Noise Objectives: Theory, Tuned Ablations, and Negative Results

Kunal

Technical report (draft)

Abstract

I implement a variance-reduced optimizer with adaptive coordinate scaling (“Coordinate SVRG”) in PyTorch and JAX, validate both against an independent NumPy oracle, and evaluate it on synthetic low signal-to-noise finite-sum problems built to resemble noisy tick data. I prove a variance bound that explains the method’s behaviour, a staleness-controlled variant of it for the adaptive engine, a numerical-safety property, and a short corollary explaining when diagonal scaling helps. Against learning-rate-tuned Adam and SGD with momentum (constant and cosine-decay), variance reduction reaches float64 round-off on well-scaled convex problems; adaptive scaling helps only when features are badly scaled; and on a noisy non-convex fit the more precisely optimised model generalises *worse*. There is no wall-clock win over a direct solve at the sizes tested, and no benefit for walk-forward signal tracking. All data are synthetic, all models small, and the PyTorch and JAX engines did not produce the reported numbers; the report states these limits explicitly.

1 Introduction

Stochastic gradient methods stall on noisy finite-sum objectives because per-sample gradients do not vanish at the optimum, so mini-batch variance is bounded below. Variance-reduced methods such as SVRG [1] remove that floor with a control variate built from a periodic snapshot. Quantitative-finance estimation problems (calibrating linear and generalised-linear signal models on noisy tick data) are finite sums with very low signal-to-noise, which makes the question natural: does variance reduction, combined with Adam-style coordinate scaling [2], help there?

Contributions.

1. A reference implementation in PyTorch and JAX with four enforced invariants (Section 2), tested step-for-step against an independent NumPy oracle, with adversarial-input fuzz tests.
2. Four short results (Section 3): a variance bound (Lemma 2), a staleness-controlled version valid for the adaptive engine (Proposition 1), a numerical-safety property (Proposition 2), and a preconditioning corollary (Proposition 3) that predicts when adaptive scaling helps.
3. A learning-rate-tuned ablation with held-out seeds and paired bootstrap intervals across ten experiments, including negative and null results.

I make no claim about real markets, deep networks, or large-scale wall-clock performance.

2 Method

Let $F(w) = \frac{1}{n} \sum_{i=1}^n f_i(w)$. Given a snapshot $\tilde{w}$ with full gradient $\mu = \nabla F(\tilde{w})$ and a mini-batch B , the variance-reduced gradient is

$$\hat{g}(w) = g_B(w) - g_B(\tilde{w}) + \mu, \quad g_B = \frac{1}{|B|} \sum_{i \in B} \nabla f_i.$$

The engine then applies coordinate-wise momentum and second-moment tracking,

$$m \leftarrow \beta_1 m + (1 - \beta_1) \hat{g}, \quad v \leftarrow \beta_2 v + (1 - \beta_2) \hat{g}^2, \quad w \leftarrow w - \eta \text{clip}\left(\frac{m/(1 - \beta_1^t)}{\max\{\sqrt{v/(1 - \beta_2^t)}, \varepsilon_{\text{floor}}\}}, \pm c\right).$$

Four invariants are enforced and tested: (1) the variance of $\hat{g}$ vanishes as $w \rightarrow \tilde{w}$; (2) $g_B(w)$ and $g_B(\tilde{w})$ use the *same* batch (one batch identifier is passed to both evaluations); (3) in PyTorch, parameters, snapshot, μ , m , v live in contiguous flat buffers; (4) gradients are sanitised and bounded and the denominator is *clamped*, not offset, so no division can produce NaN or Inf.

3 Theory

Let each f_i have L_i -Lipschitz gradient and write $\bar{L}^2 = \frac{1}{n} \sum_i L_i^2$.

Lemma 1 (Unbiasedness). *For any snapshot, $\mathbb{E}_B[\hat{g}(w)] = \nabla F(w)$.*

Proof. $\mathbb{E}_B g_B(w) = \nabla F(w)$ and $\mathbb{E}_B g_B(\tilde{w}) = \mu$. □

Lemma 2 (Variance bound). *For uniform mini-batches of size b (with or without replacement),*

$$\mathbb{E} \|\hat{g}(w) - \nabla F(w)\|^2 \leq \frac{\bar{L}^2}{b} \|w - \tilde{w}\|^2.$$

For least squares the error is exactly $(H_B - H)(w - \tilde{w})$, so scaling the displacement by s scales the variance by exactly s^2 .

Proof. With $Y_i = \nabla f_i(w) - \nabla f_i(\tilde{w})$, $\hat{g} - \nabla F = \bar{Y}_B - \bar{Y}$, a sample-mean error. Its variance is σ^2/b (times $\frac{n-b}{n-1} \leq 1$ without replacement) with $\sigma^2 = \frac{1}{n} \sum_i \|Y_i - \bar{Y}\|^2 \leq \frac{1}{n} \sum_i \|Y_i\|^2 \leq \bar{L}^2 \|w - \tilde{w}\|^2$. □

Plain mini-batch SGD has no such control: at a stationary point of a noisy problem, per-sample gradients do not vanish, so its variance has a positive floor. The variance of $\hat{g}$ depends on the snapshot distance, not on the data noise; this is why the snapshot must be refreshed.

Proposition 1 (Staleness-controlled variance). *If the normalised update is clipped to $\pm c$ per coordinate, then K steps after a snapshot,*

$$\mathbb{E} \|\hat{g} - \nabla F\|^2 \leq \frac{\bar{L}^2 d}{b} (\eta c K)^2.$$

Proof. Each coordinate moves at most ηc per step, so $\|w - \tilde{w}\|_2^2 \leq d(\eta c K)^2$; apply Lemma 2. □

This holds for any adaptive normalisation provided the step is clipped. It is very loose: across snapshot intervals from 8 to 1024, measured variance was at most 2.2×10^{-4} of the bound.

Proposition 2 (Numerical safety). *Let $G = \frac{1}{4} \sqrt{\text{realmax}}$ for the working precision. After sanitising $\text{NaN} \rightarrow 0$ and $\pm\infty \rightarrow \pm G$ and clipping to $[-G, G]$, $|m_t| \leq G$ and $0 \leq v_t \leq G^2$ by induction; the denominator is at least $\varepsilon_{\text{floor}} > 0$ and the final ratio is clipped, so every step is finite with $|\Delta w_j| \leq \eta c$. In single precision the unclipped quotient is itself finite whenever $1 - \beta_1 \geq 10^{-4}$.*

This is checked by randomised adversarial tests (zeros, denormals, $10^{\pm 38}$, $\pm\infty$, NaN) in float32 and float64.

Proposition 3 (Frozen diagonal preconditioner). *Let $D \succ 0$ be diagonal and fixed, and consider $w \leftarrow w - \eta D^{-1} \hat{g}$. If $\tilde{f}_i(u) = f_i(D^{-1/2}u)$ are L_D -smooth and $\tilde{F}$ is γ_D -strongly convex, the Johnson–Zhang theorem [1] holds verbatim with (L_D, γ_D) : for step $\eta < 1/(2L_D)$ and m inner steps, $\mathbb{E}[F(\tilde{w}_s) - F(w^*)] \leq \alpha_D^s (F(\tilde{w}_0) - F(w^*))$ with*

$$\alpha_D = \frac{1}{\gamma_D \eta (1 - 2L_D \eta) m} + \frac{2L_D \eta}{1 - 2L_D \eta}.$$

Proof. Put $u = D^{1/2}w$. Then $\nabla \tilde{f}_i(u) = D^{-1/2} \nabla f_i(w)$, so the SVRG step in u is $D^{1/2}$ applied to the preconditioned step in w : the iterations coincide up to a linear change of coordinates (verified numerically in the test suite), with equal function values. $\square$

The rate depends on $\kappa_D = L_D/\gamma_D$. For least squares with Jacobi scaling $D = \text{diag}(X^\top X/n)$, κ_D is invariant to rescaling feature columns, and Jacobi scaling is within a factor of the dimension of the best diagonal scaling [5]. On the badly scaled problem below, κ falls from 1.1×10^7 to 82; on equal-variance features it does not move ($81 \rightarrow 82$).

Remark. The engine’s preconditioner $\text{diag}(\sqrt{\hat{v}_t})$ is not frozen and tracks gradient scale rather than Hessian-diagonal scale. The proposition covers the frozen idealisation; the link to the adaptive engine is empirical consistency, not a theorem. **I prove no convergence theorem for the adaptive engine itself.** Adam-type methods can fail to converge in general [3], and normalisation keeps the step near η per coordinate even as gradients shrink, which explains the constant-rate plateau seen in Section 5.

4 Experimental protocol

All results use a NumPy reference implementation of the update rule; the PyTorch and JAX engines are tested against it but *did not produce the numbers reported here*. Data come from a simulator: AR(1) features, a hidden alpha with drift and regime switches, Gaussian noise at three levels, heavy-tailed toxic-flow shocks and arbitrage jumps (empirical SNR about -4 dB in the default regime). Every method receives its own learning-rate sweep on tuning seeds and is scored on disjoint held-out seeds. Budgets count sample-gradient evaluations; SVRG is charged for double gradients and every full-gradient snapshot pass. Baselines are Adam (with the same clamped denominator) and SGD with momentum, each with constant and cosine-decay learning rates. Differences are reported in decades of $\log_{10}(\text{final gap})$ with 95% paired bootstrap intervals over seeds and win counts; no multiple-comparison correction is applied. Where one method sits at float64 round-off ($\approx 10^{-31}$), the *magnitude* of a difference is not meaningful, only the win count.

5 Results

5.1 Variance reduction dominates on well-scaled convex problems

On least squares (volatile regime, Table 1) Coordinate SVRG reaches a median gap of 4.1×10^{-31} , at float64 round-off, and beats the best tuned baseline (SGD + momentum, cosine lr, 9.7×10^{-8}) on 20/20 held-out seeds. Across the three noise regimes the best baseline stops at 8.3×10^{-8} to 2.8×10^{-7} . On ℓ_2 -regularised logistic regression for the sign of the return the win count is 12/12.

Method	tuned lr	median final gap	evals to target
SGD + momentum	1.8×10^{-3}	7.8×10^{-6}	not reached
SGD + momentum, cosine lr	4.2×10^{-3}	9.7×10^{-8}	not reached
Adam (clamped)	3.2×10^{-4}	4.2×10^{-6}	not reached
Adam (clamped), cosine lr	7.5×10^{-4}	9.7×10^{-8}	not reached
SVRG + momentum	1.3×10^{-1}	3.9×10^{-31}	72 448
Coordinate SVRG (ours)	1.8×10^{-3}	4.1×10^{-31}	85 376
Coordinate SVRG, cosine lr (ours)	1.0×10^{-2}	4.0×10^{-31}	85 376

Table 1: Least squares, volatile regime. Median final loss gap and sample-gradient evaluations to reach 10^{-8} of the initial gap, held-out seeds. †: best learning rate on the edge of the grid.

Method	tuned lr	median final gap	evals to target
SGD + momentum	3.2×10^{-4}	5.3×10^{-2}	not reached
SGD + momentum, cosine lr	1.0×10^{-2}	1.7×10^{-2}	not reached
Adam (clamped)	7.5×10^{-4}	4.3×10^{-3}	not reached
Adam (clamped), cosine lr	1.0×10^{-2}	5.9×10^{-6}	not reached
SVRG + momentum	1.0×10^{-2}	2.2×10^{-2}	not reached
Coordinate SVRG (ours)	1.0×10^{-2}	6.8×10^{-4}	not reached
Coordinate SVRG, cosine lr (ours)	5.6×10^{-2}	3.5×10^{-11}	348 288

Table 2: Least squares with feature scales spread over three decades. Same protocol as Table 1.

5.2 Adaptive scaling helps only when features are badly scaled

With column scales spread over three decades (Table 2), Coordinate SVRG at a constant rate beats SVRG with plain momentum by -1.5 decades (95% CI [-2.0, -1.1], 20/20 seeds), yet plateaus at a median gap of 6.8×10^{-4} , as anticipated in Section 3. Adding cosine decay removes the plateau (median 3.5×10^{-11}) and beats the best tuned baseline, Adam with cosine decay (5.9×10^{-6}), by -5.3 decades (CI [-6.6, -4.9], 20/20 seeds). On well-scaled data the adaptive component does not help and, at constant rate on the larger problem, hurts. Proposition 3 predicts this pattern through κ_D .

5.3 The variance law along a real trajectory

Along one run (Figure 3), mini-batch SGD variance stays between 1.7 and 2.5, while the variance of $\hat{g}$ falls from 2.6×10^{-1} to 9.9×10^{-16} . In all 25 probes, each at worst-case snapshot staleness, measured variance was within the Lemma 2 bound (100%) and the Proposition 1 bound (100%).

5.4 Non-convex fit: sharper stationarity, worse generalisation

For a one-hidden-layer tanh network the squared full-gradient norm of Coordinate SVRG with cosine decay is -1.3 decades from the best baseline (8/8 seeds). On a held-out random split, however, with learning rates tuned on training loss, its test loss is 0.1445 against 0.1317 for SGD + momentum, cosine lr (paired difference +0.0135, 95% CI [+0.0106, +0.0150]; the noise floor is 0.125), while its training loss is lower (0.1079 against 0.1177). It fits the training noise harder. Tuning on validation loss could change the ranking. For held-out logistic regression all methods are indistinguishable (test log-loss 0.68775 to 0.68776, about 54% accuracy).

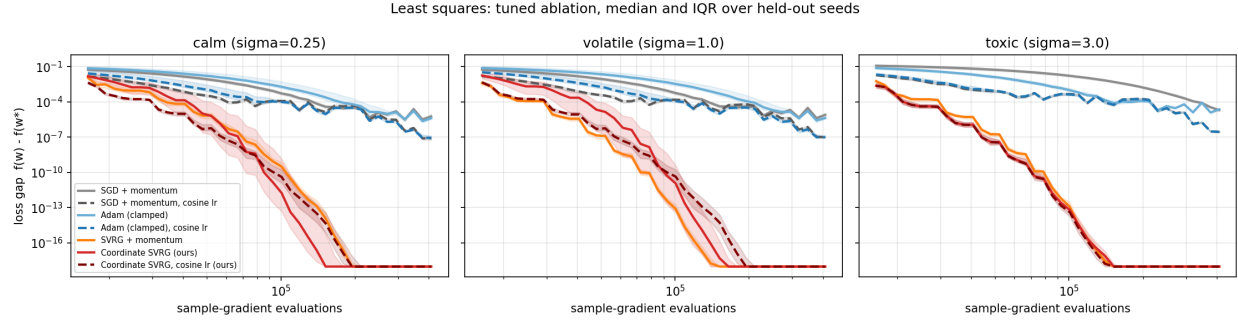


Figure 1: Least squares, three noise regimes: loss gap against sample-gradient evaluations (median and IQR over held-out seeds). Curves are clamped at 10^{-18} for plotting.

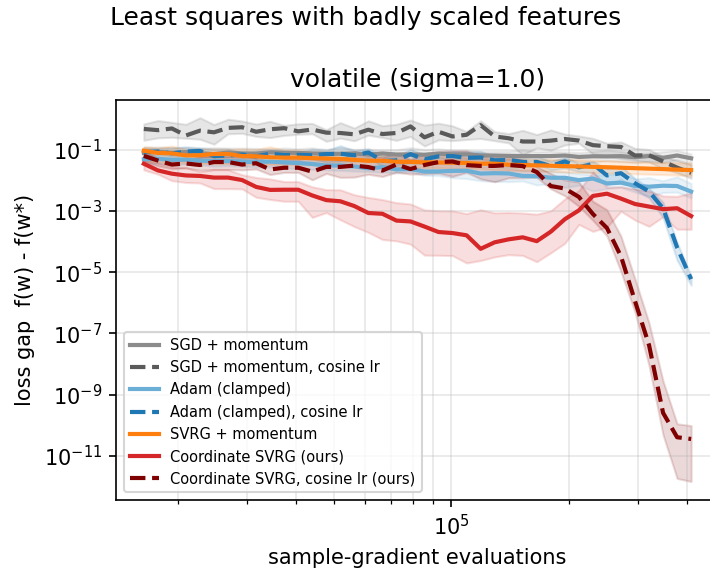


Figure 2: Badly scaled features. Only variance reduction with adaptive scaling and a decaying rate reaches the target.

5.5 Snapshot interval and wall-clock

Evaluations to the target are U-shaped in the snapshot interval K : best at $K = 64$ (60,800 evaluations), within a factor 2.1 for $16 \leq K \leq 256$, $2.2\times$ worse at $K = 8$ and $5.6\times$ worse at $K = 1024$ (Figure 4). On a larger problem ($n = 40000$, $d = 200$) SVRG with momentum reaches the target in 0.08 s on one NumPy core, whereas the normal equations take 0.06 s and `lstsq` 0.48 s: a direct solve still matches SVRG at this size, and the iterative timing excludes the cost of tuning the learning rate.

5.6 Walk-forward signal tracking: no benefit

In a rolling re-estimation of a drifting alpha under a compute budget, every method’s out-of-sample correlation with the clean signal differs from closed-form OLS by at most 0.0054, against a spread across datasets of about 0.08; paired intervals often exclude zero but the effects are negligible, and

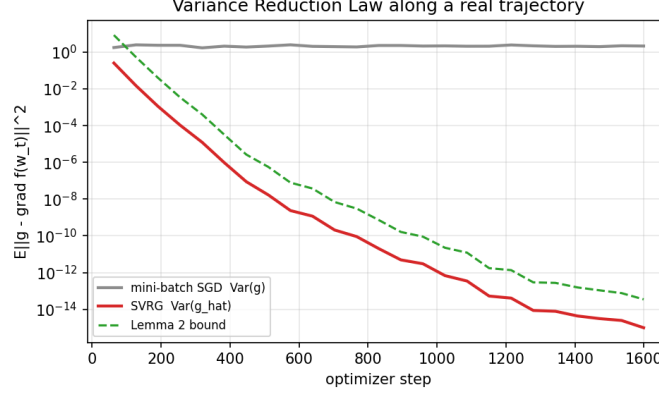


Figure 3: Gradient variance along the trajectory, with the Lemma 2 bound.

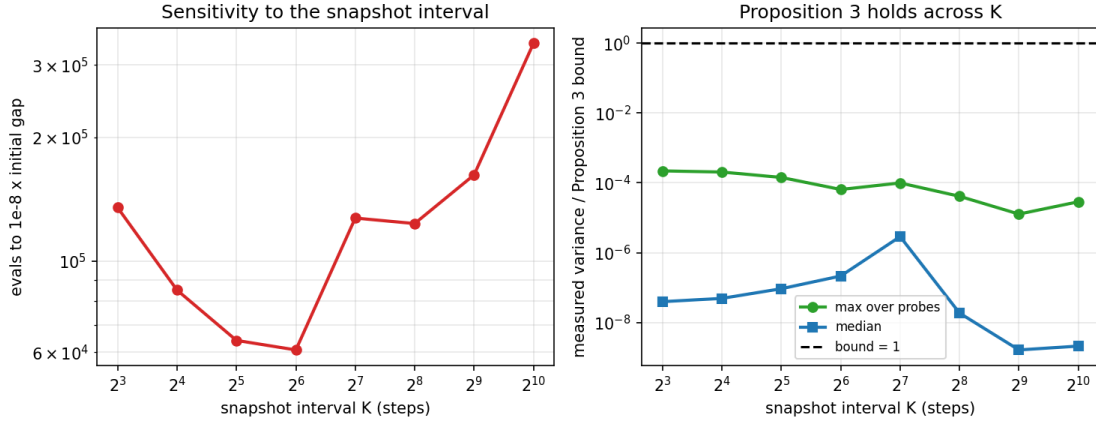


Figure 4: Sensitivity to the snapshot interval, and the Proposition 1 ratio across intervals.

SVRG never beats Adam. The problem is too small for the choice of optimizer to matter.

6 Limitations and threats to validity

- **Synthetic data.** The simulator generates what the models assume. Nothing here is evidence of real-market performance.
- **Convex bias.** Most experiments are convex, the best case for SVRG; variance reduction is known to help far less in deep learning [4]. One small non-convex network is not evidence about deep models.
- **Scale.** The largest problem has $n = 40000$, $d = 200$, where a direct solve is as fast as any iterative method.
- **Engines not executed by the author’s sandbox.** The PyTorch and JAX engines are validated by tests written against the oracle, to be run on the reader’s machine; the reported numbers are from the NumPy oracle.
- **No convergence proof for the adaptive engine;** the cosine-decay results are empirical.

- **Statistics.** Seed counts range from 3 to 20; intervals are bootstrap intervals over seeds; no multiple-comparison correction; learning rates are tuned on training loss in the non-convex and logistic held-out checks.
- **Tuning cost** is excluded from the iterative wall-clock times.

7 Reproducibility

The repository fixes every random stream from one seed. `make test` runs the invariant, oracle, differential and fuzz tests; `make experiments` regenerates all results and figures; `make report` rebuilds this document, whose numbers are macros generated from `results/experiments.json`. A continuous-integration workflow runs the tests on CPU PyTorch and JAX.

References

- [1] R. Johnson, T. Zhang. Accelerating stochastic gradient descent using predictive variance reduction. *NeurIPS*, 2013.
- [2] D. Kingma, J. Ba. Adam: a method for stochastic optimization. *ICLR*, 2015.
- [3] S. Reddi, S. Kale, S. Kumar. On the convergence of Adam and beyond. *ICLR*, 2018.
- [4] A. Defazio, L. Bottou. On the ineffectiveness of variance reduced optimization for deep learning. *NeurIPS*, 2019.
- [5] A. van der Sluis. Condition numbers and equilibration of matrices. *Numerische Mathematik* 14, 1969.